AICE2005 - Systematic Design

Formal Specification and Verification

Asieh Salehi Fathabadi
University of Southampton
2025-2026

© University of Southampton

Course Introduction

- Lecture 1: Introduction to Formal Methods
- Lecture 2: Formal Specification Techniques
- Lecture 3: Mathematical Foundations
- Lecture 4: Formal Verification Methods
- Lecture 5: Tools and Applications

© University of Southampton

LECTURE 5:
Tools and Applications



266001

© University of Southampton

Lecture 5 Overview

- Tool Landscape Overview
- B/Event-B Tools (Rodin Platform, ProB, Atelier B)
- Event-B Case Study (Automatic Level Crossing)
- Course Summary and Assessment

Learning Objectives:

- Apply Event-B to simple realistic system modeling
- Understand industrial integration practices

© University of Southampton

Recap: Verificarion and Validation

- **Validation vs Veirificaiton**
  - Building the right system - Building the system Right
- **Model Checking for verification**
  - Explore all possible state to check safety properties
- **Animation for validation**
  - Validate specific scenario with doamin expert
- **Proof Obligations**
  - Theorem proving in Event-B

© University of Southampton

Event-B Tool Ecosystem

Rodin Platform (Event-B):

- Type: Eclipse-based IDE for Event-B
- Features: Modeling, proof management, refinement support
- Provers: Automatic (PP, ML), Interactive (manual proof)
- Extensions: ProB integration, code generation, documentation

Key Rodin Features:

- Event-B Editor: Model creation and editing | Syntax checking
- Proof Obligation View: Track proof progress | Visual proof status, statistics
- Prover UI: Interactive theorem proving | Step-by-step proof construction
- Refinement Support: Multi-level development | Automatic refinement checking

© University of Southampton

Event-B Tool Ecosystem

ProB (Model Checker and Animator):

- Animation: Interactive model exploration
- Model Checking: Automatic verification of properties

Atelier B (Classical B Method):

- Focus: Industrial B Method development
- Strengths: Mature tool, proven track record
- Applications: Safety-critical systems (railway, avionics)

© University of Southampton

NEWS

Home | InDepth | Israel-Gaza war | War in Ukraine | Climate | UK | World | Business | Politics | Culture

World | Africa | Asia | Australia | Europe | Latin America | Middle East | US & Canada



Watch: Train smashes into truck at Dutch level crossing

CCTV footage shows the dramatic moment a train crashed into a truck at a level crossing in the Netherlands.

The vehicle can be seem manoeuvring forwards and backwards on the railway line before the barriers descend. It is then hit at speed.

Five people sustained minor injuries but nobody was seriously hurt in the incident in Meteren, train operator Prorail said.

Europe · 31 October

https://www.bbc.co.uk/news/videos/ced65d0gl2eo?app-referrer=search

© University of Southampton

Automatic Level Crossing Control System

System Description

A railway level crossing protects road users from trains by lowering barriers when a train is approaching and raising them after the train passes. The system includes:

- **Train sensors** (detect train state)
- **Barriers** (open or closed)
- **Signal lights** (green for cars to pass, red when barrier closed)
- **Controller** (manages synchronization)

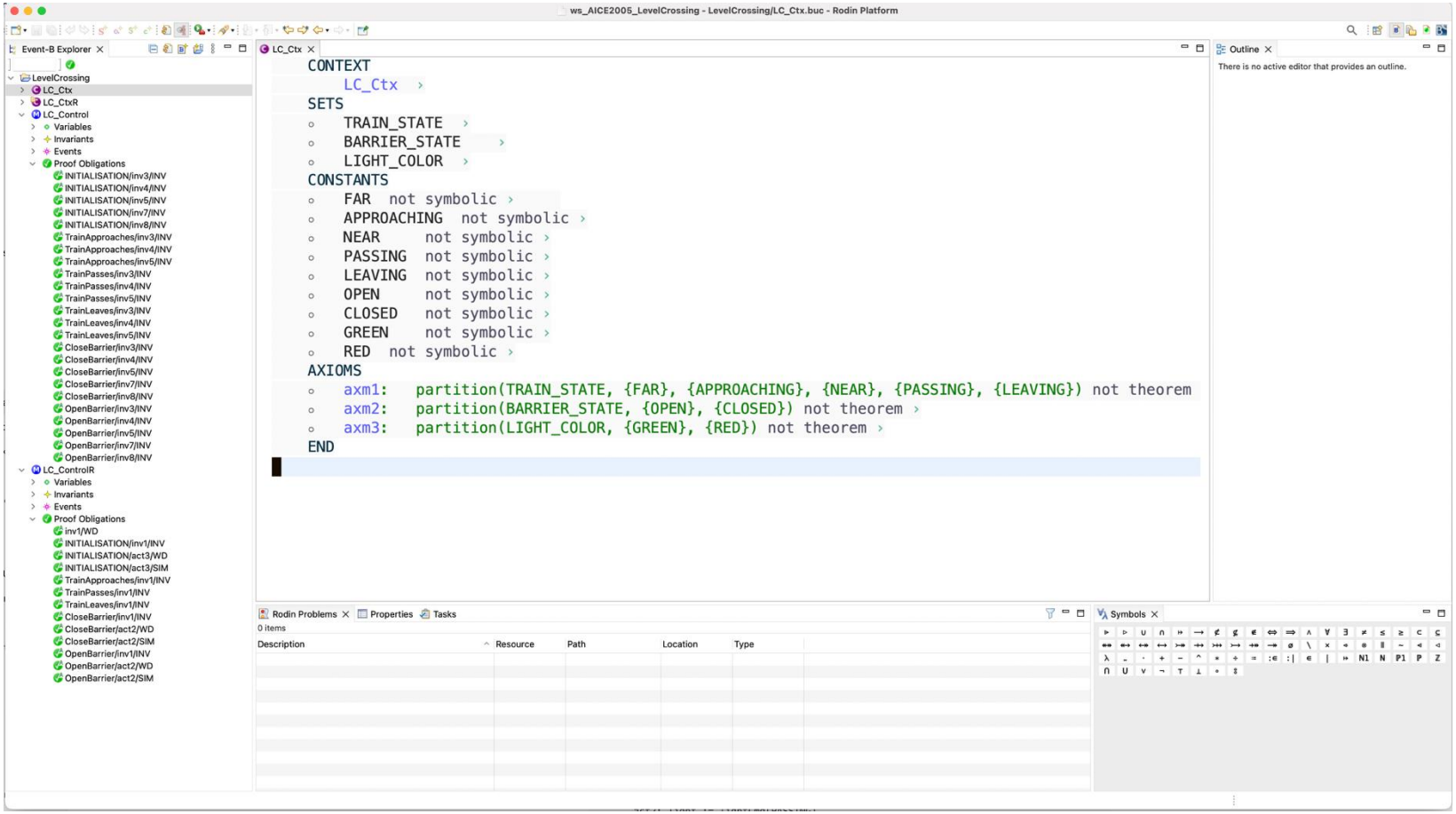
Let's think and reflect

Identify Safety Requirements

ID	Safety Requirement	Meaning
SR1	Barriers shall be **closed** whenever a train is near or crossing.	Prevents collisions
SR2	Barriers shall be **open** only when no train is detected.	Allows road traffic safely
SR3	Signal lights must be **red** whenever barriers are closed.	Warns vehicles
SR4	System must never have barriers open and train detected simultaneously.	Core invariant safety property

© University of Southampton

Event-B Model
Rodin tool experience (demo)



[Play Here]

© University of Southampton
University of Southampton

LC_Ctx

```
CONTEXT
    LC_Ctx ›
SETS
    TRAIN_STATE ›
    BARRIER_STATE ›
    LIGHT_COLOR ›
CONSTANTS
    FAR not symbolic ›
    APPROACHING not symbolic ›
    NEAR not symbolic ›
    PASSING not symbolic ›
    LEAVING not symbolic ›
    OPEN not symbolic ›
    CLOSED not symbolic ›
    GREEN not symbolic ›
    RED not symbolic ›
AXIOMS
    axm1: partition(TRAIN_STATE, {FAR}, {APPROACHING}, {NEAR}, {PASSING}, {LEAVING}) not theorem
    axm2: partition(BARRIER_STATE, {OPEN}, {CLOSED}) not theorem ›
    axm3: partition(LIGHT_COLOR, {GREEN}, {RED}) not theorem ›
END
```


© University of Southampton

LC_Ctx | LC_Control

```
MACHINE
    LC_Control  ›
SEES
◦    LC_Ctx
VARIABLES
◦   train    ›
◦   barrier  ›
◦   light    ›
INVARIANTS
◦   inv1:   train ∈ TRAIN_STATE not theorem ›
◦   inv2:   barrier ∈ BARRIER_STATE not theorem ›
◦   inv3:   train ∈ {NEAR, PASSING} ⇒ barrier = CLOSED  not theorem ›SR1
◦   inv4:   barrier = OPEN ⇒ train ∈ {APPROACHING, FAR, LEAVING}  not theorem ›SR2
◦   inv5:   ¬(barrier = OPEN ∧ train ∈ {NEAR, PASSING}) not theorem ›SR4 (redundant from inv4, but explicit)
◦   inv6:   light ∈ LIGHT_COLOR not theorem ›
◦   inv7:   barrier = CLOSED ⇒ light = RED not theorem ›SR3
◦   inv8:   barrier = OPEN ⇒ light = GREEN  not theorem ›
EVENTS
◦   INITIALISATION:  not extended ordinary ›
    THEN
    ◦   act1:   train = FAR ›
    ◦   act2:   barrier = OPEN ›
    ◦   act3:   light = GREEN ›
    END

◦   TrainApproaches:     not extended ordinary ›
    WHERE
    ◦   grd1:   train = FAR not theorem ›
    THEN
    ◦   act1:   train = APPROACHING ›
    END

◦   TrainPasses:     not extended ordinary ›
    WHERE
    ◦   grd1:   train = NEAR not theorem ›
    THEN
    ◦   act1:   train = PASSING ›
    END

◦   TrainLeaves:     not extended ordinary ›
    WHERE
    ◦   grd1:   train = PASSING not theorem ›
    THEN
    ◦   act1:   train = LEAVING ›
    END

◦   CloseBarrier:    not extended ordinary ›
    WHERE
    ◦   grd1:   barrier = OPEN  not theorem ›
    ◦   grd2:   train = APPROACHING not theorem ›
    THEN
    ◦   act1:   barrier = CLOSED ›
    ◦   act2:   light = RED ›
    ◦   act3:   train = NEAR ›
    END

◦   OpenBarrier:     not extended ordinary ›
    WHERE
    ◦   grd1:   barrier = CLOSED not theorem ›
    ◦   grd2:    train = LEAVING not theorem ›
    THEN
    ◦   act1:   barrier = OPEN ›
    ◦   act2:   light = GREEN ›
    ◦   act3:   train = FAR ›
    END

END
```


© University of Southam
University of Southampton

LC_Ctx | LC_Control | LC_CtxR ×

```
CONTEXT
    LC_CtxR     ›Introduce a control function from train state to commanded barrier state.
EXTENDS
◦    LC_Ctx
CONSTANTS
◦   lightCmd     not symbolic ›
AXIOMS
◦   axm2:   lightCmd ∈ TRAIN_STATE → LIGHT_COLOR not theorem ›
◦   axm3:   lightCmd(FAR) = GREEN not theorem ›
◦   axm4:   lightCmd(NEAR) = RED not theorem ›
◦   axm5:   lightCmd(PASSING) = RED not theorem ›
◦   axm6:   lightCmd(APPROACHING) = GREEN not theorem ›
◦   axm7:   lightCmd(LEAVING) = RED not theorem ›
END
```


© University of Southampton

LC_Ctx | LC_Control | LC_CtxR | LC_ControlR

```
MACHINE
    LC_ControlR     ›
REFINES
◦    LC_Control
SEES
◦    LC_CtxR
VARIABLES
◦   train     ›
◦   barrier   ›
◦   light     ›
INVARIANTS
◦   inv1:   light = lightCmd(train) not theorem ›gluing inv
EVENTS
◦   INITIALISATION:  not extended ordinary ›
    THEN
    ◦   act1:   train = FAR ›
    ◦   act2:   barrier = OPEN ›
    ◦   act3:   light = lightCmd(FAR) ›
    END

◦   TrainApproaches:     extended ordinary ›
    REFINES
    ◦    TrainApproaches
    WHERE
    ◦   grd1:   train = FAR not theorem ›
    THEN
    ◦   act1:   train = APPROACHING ›
    END

◦   TrainPasses:     extended ordinary ›
    REFINES
    ◦    TrainPasses
    WHERE
    ◦   grd1:   train = NEAR not theorem ›
    THEN
    ◦   act1:   train = PASSING ›
    END

◦   TrainLeaves:     extended ordinary ›
    REFINES
    ◦    TrainLeaves
    WHERE
    ◦   grd1:   train = PASSING not theorem ›
    THEN
    ◦   act1:   train = LEAVING ›
    END

◦   CloseBarrier:    not extended ordinary ›
    REFINES
    ◦    CloseBarrier
    WHERE
    ◦   grd1:   barrier = OPEN  not theorem ›
    ◦   grd2:   train = APPROACHING not theorem ›
    THEN
    ◦   act1:   barrier = CLOSED ›
    ◦   act2:   light = lightCmd(NEAR) ›
    ◦   act3:   train = NEAR ›
    END

◦   OpenBarrier:     not extended ordinary ›
    REFINES
    ◦    OpenBarrier
    WHERE
    ◦   grd1:   barrier = CLOSED not theorem ›
    ◦   grd2:    train = LEAVING not theorem ›
    THEN
    ◦   act1:   barrier = OPEN ›
    ◦   act2:   light = lightCmd(FAR) ›
    ◦   act3:   train = FAR ›
    END

END
```


© University of Southampton

Summery

- **Event-B tool (Rodin) overview**
- **Automatic Level Crossing Control System:**
  - Formal modelling and Formal V&V overview

Let's read and explore

  - Modeling in Event-B: System and Software Engineering
    by Jean-Raymond Abrial

https://wiki.event-b.org/index.php/Main_Page

© University of Southampton